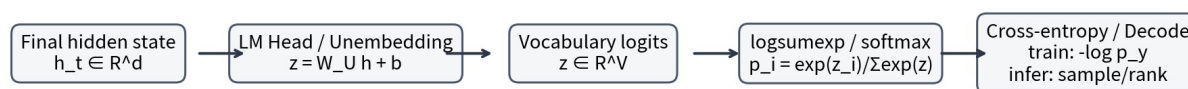


LM Head / Unembedding / Vocabulary Softmax

深度分析 2026

Hidden State $\rightarrow$ Vocabulary Logits $\rightarrow$ Cross-Entropy / Decoding



训练时真正需要的是 log-softmax / cross-entropy；推理时 softmax 常可延迟到候选筛选之后。

算法原理 • Weight Tying • Softmax Bottleneck • 大词表系统成本 • Vocab Parallel • 数值稳定 • Gotchas •
Know-how • Know-why

研究与核验日期：2026-08-14 | 经典结论与 2026 新兴研究分层标注

0. Executive Summary : LM Head 为什么值得单独研究

在 Decoder-only LLM 中，Transformer 主干最终只产生一个 d 维 hidden state；真正把这个连续向量变成“下一个 token”的，是 LM Head / Unembedding。最简单形式是一个 $V \times d$ 的线性映射： $z = W_U h + b$ ，其中 V 是词表大小。随后 cross-entropy 使用 logsumexp 把 V 个 logits 归一化为概率。

这个模块看起来只是“一层 Linear”，但它同时决定：① 词表参数成本；② hidden space 与 token space 的几何接口；③ 训练监督信号如何反传回主干；④ 大词表下的 GEMM 与显存成本；⑤ Tensor Parallel 如何划分；⑥ weight tying 是否把输入和输出任务强耦合；⑦ logits 是否出现数值发散；⑧ softmax 的低秩表达限制。

现代 LLM 的趋势并不是简单地“统一使用 tied LM Head”。经典工作证明 weight tying 可显著减参并改善若干语言模型；但最新受控研究开始指出，深层 Transformer 中 output-gradient domination、input/output geometry 冲突等问题可能让 untied 或更柔性的 tying 更有吸引力。[R1][R15][R16]

另一个常见误区是把 softmax 当作推理中最大的成本。训练时， $V \times d$ 的 dense projection 和 cross-entropy 都必须处理真实 target；而推理解码时，即便最终只采样 Top-k，通常仍先需要产生足够的 vocabulary logits，除非采用 shortlist / hierarchical / approximate-output 等专门结构。

核心视角：LM Head 不是“最后一层”而已。更准确的理解是：它是 residual representation 与离散 token ontology 之间的双向接口，也是监督信号进入模型的最后一道几何瓶颈。

问题	最短答案	工程后果
为什么参数可能很大？	参数量约 $V \times d$ ；若不 tying，还要在 input embedding 之外再付一份。	大词表 + 大 d 时，embedding/head 可占数亿到十亿参数。
为什么不用显式 softmax 再算 loss？	CrossEntropy = LogSoftmax + NLL，可直接稳定计算。	避免 exp 溢出和不必要概率张量。
为什么 weight tying 有吸引力？	减少 $V \times d$ 参数，并把读/写 token 空间耦合。	小模型和大词表模型尤其显著。
为什么 tying 不是免费午餐？	输入表示与输出分类器收到不同梯度目标。	可能形成 output-biased shared embedding。
为什么大词表需要 vocab parallel？	完整 logits 张量和 W_U 太大。	按 V 维分片，分布式计算 max/sum-exp/target logit。
Softmax bottleneck 是什么？	log-probability matrix 的表达 rank 受 hidden dimension 限制。	极端情况下限制多模态 next-token 分布表达。

目录

1. 定义与完整数学链路
2. LM Head 的形状、参数量与 FLOPs
3. Softmax / LogSoftmax / Cross-Entropy
4. 梯度：监督信号如何从 Vocabulary 回到 Hidden State
5. Weight Tying：经典动机、收益与代价
6. Softmax Bottleneck 与表示能力
7. 大词表：Vocabulary Size $\times$ Hidden Size 的系统权衡
8. Adaptive / Hierarchical / Sampled Softmax 的历史与现代位置
9. 分布式训练：Vocabulary Parallel LM Head
10. 推理：logits、temperature、top-k/top-p 与实际成本
11. 数值稳定：logsumexp、logit drift、soft-capping、z-loss
12. Quantization / Mixed Precision / FP8
13. Special Tokens、Tokenizer 与 LM Head 的联合 Gotchas
14. 50 个 Gotchas
15. Know-how：从零实现与生产检查清单
16. Know-why：核心因果原理表
17. Monitoring 与 Debug Playbook
18. Ablation / Benchmark Matrix
19. 2024-2026 新研究与开放问题
20. PyTorch Reference Implementation
21. 面试 / 推导题
22. 参考文献

1. 定义与完整数学链路

设 Transformer 最后一层在位置 t 的 hidden state 为 $h_t \in \mathbb{R}^d$ ，词表大小为 V 。最常见的 LM Head 是线性分类器：

$$z_t = W_U h_t + b, \quad W_U \in \mathbb{R}^{(V \times d)}, \quad z_t \in \mathbb{R}^V$$

对 token i ：

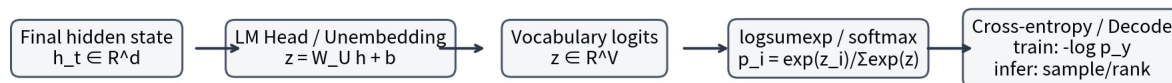
$$z_{\{t, i\}} = w_i^T h_t + b_i$$

因此每个 vocabulary row w_i 都可以理解为“输出 token i 的方向 / classifier prototype”。如果没有 bias，则 token 排序完全由 hidden state 与各 row 的点积决定。

训练目标通常是 next-token cross-entropy。目标 token 为 y 时：

$$L_t = -\log \text{softmax}(z_t)_y = -z_{\{t, y\}} + \log \sum_j \exp(z_{\{t, j\}})$$

注意：训练时通常不需要先 materialize 一个概率向量再取 log。框架直接通过稳定的 log-sum-exp 路径计算 CrossEntropyLoss。[R10]



训练时真正需要的是 log-softmax / cross-entropy；推理时 softmax 常可延迟到候选筛选之后。

图 1 LM Head 从 hidden state 到训练 loss / 推理解码的完整链路

Know-why: LM Head 的每一行不是“一个词向量的静态语义定义”。它是为了在所有上下文 hidden states 上把对应 token 的 logit 排到正确位置而优化的输出分类方向。

1.1 Unembedding 这个名字为什么重要

在 weight tying 的模型里，输入 embedding 使用 $E[\text{token}]$ 把离散 token 映射到 hidden/channel 空间；输出端常用 E^T 把 hidden state 投影回 vocabulary logits，因此社区常称其为 unembedding。这个名称强调了输入和输出是两个相反方向的接口，但并不意味着它们数学上严格互逆。

即使 $W_U = E$ ，通常也没有 $E^T E = I$ ，更没有 $\text{softmax}(Eh)$ 等于“反向恢复原 token”。因此把 weight tying 解释为 exact inverse 是一个常见误区。

2. LM Head 的形状、参数量与 FLOPs

对于 batch B、sequence length L:

$$H: [B, L, d] \quad W_U: [V, d] \quad \text{logits}: [B, L, V]$$

训练阶段若所有位置都计算 next-token loss，主要矩阵乘法约为：

$$\text{FLOPs}_{\text{forward}} \approx 2 \times B \times L \times d \times V$$

反向还要计算 dW_U 和 dH ，因此 LM Head 的训练总成本会进一步放大。模型主干越小、 V 越大，这一层越可能成为不可忽略的计算部分。

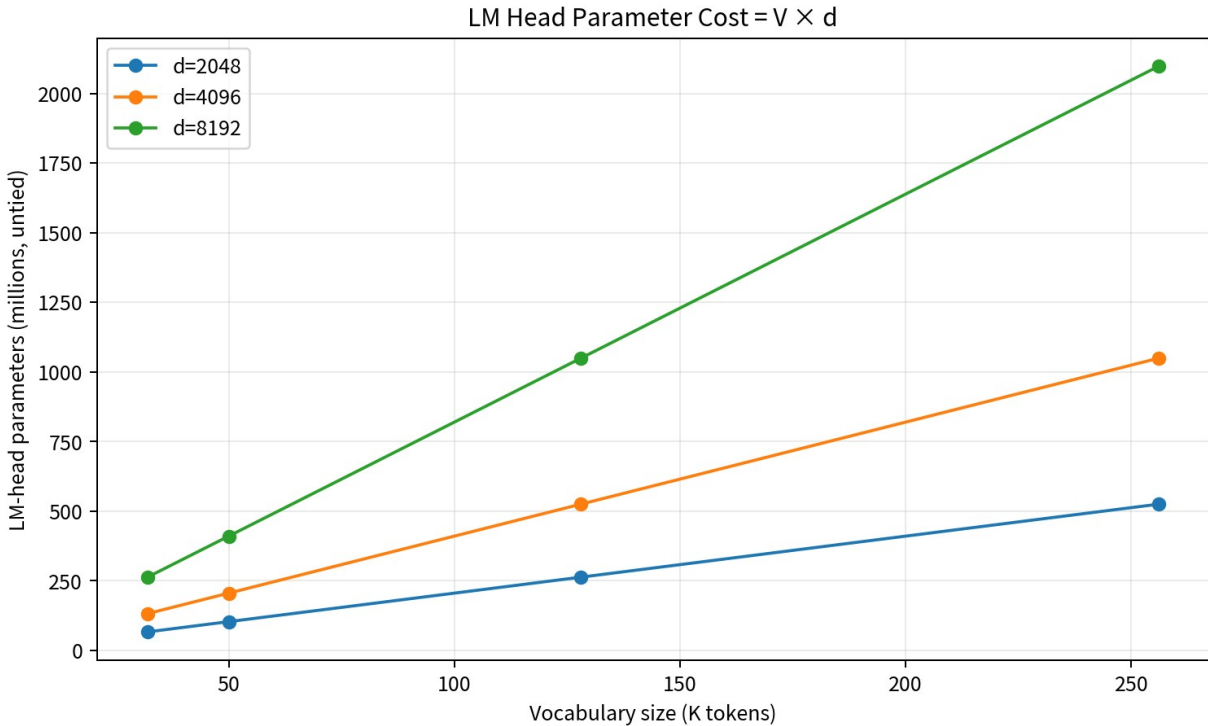


图 2 不共享 LM Head 时，参数量随 V 与 d 线性增长

Vocab V	Hidden d	W_U 参数	BF16 权重	AdamW 训练状态（粗略）
32K	4096	131M	≈0.26 GB	可达 ≈2 GB 级（取决于 master weight / optimizer dtype）
128K	4096	524M	≈1.05 GB	数 GB 级
256K	4096	1.05B	≈2.10 GB	可达十 GB 附近的参数+优化器状态级别
256K	8192	2.10B	≈4.19 GB	进一步线性增长

上表的 optimizer state 只用于建立量级直觉，真实占用取决于是否保留 FP32 master weights、Adam moments 精度、ZeRO/FSDP 分片等。不要把一个固定“每参数 N bytes”常数当成所有训练栈的真值。

2.1 Gemma 2：大词表让 embedding/head 成为架构级预算

Gemma 2 技术报告公开了 256,128 的 vocabulary，并使用 tied embedding。其 2B 配置 embedding 参数约 5.90 亿，技术报告明确指出大词表是 embedding 参数较大的原因。[R11] 这说明：Tokenizer 的 vocabulary 设计会直接回流到参数预算、通信与推理成本。

3. Softmax / LogSoftmax / Cross-Entropy

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

直接计算 $\exp(z)$ 容易溢出。利用 shift invariance:

$$\text{softmax}(z) = \text{softmax}(z - c), \quad \text{任意标量 } c$$

通常取 $c = \max(z)$:

$$\text{logsumexp}(z) = m + \log \sum_j \exp(z_j - m), \quad m = \max_j z_j$$

这正是稳定 cross-entropy 的核心。

3.1 Cross-Entropy 的梯度为什么漂亮

$$\partial L / \partial z_i = p_i - 1[i=y]$$

它直接表达“预测概率 - one-hot 目标”。正确 token 收到负梯度推动其 logit 上升；所有其他 token 按当前概率收到正梯度，概率越高的错误 token 被压得越强。

$$\partial L / \partial h = W^T (p - \text{one_hot}(y))$$

因此 hidden state 的监督方向是所有输出 token rows 的加权组合。LM Head 的几何结构决定什么样的错误信号能被送回 residual stream。

Gotcha: Softmax 概率对 logits 的共同平移不敏感，但训练数值并不因此对 logit 绝对值完全无所谓。过大的共同偏移/范数仍会影响有限精度、优化器动态、soft-capping 或正则项。

3.2 Temperature 的本质

$$p_i(T) = \text{softmax}(z_i / T)$$

$T < 1$ 放大 logit 间隔，使分布尖锐； $T > 1$ 缩小间隔，使分布平坦。Temperature 不改变 logit 排序，但改变概率比：

$$p_i / p_j = \exp((z_i - z_j) / T)$$

因此 temperature 是 gap rescaling，不是“给模型增加知识/随机性”。随机性来自后续 sampling。

4. 梯度：Vocabulary 监督如何回到 Hidden State

对单个 token，输出层梯度可以分成两条：

- 对 LM Head： $\partial L / \partial W_U = (p - y_{\text{onehot}}) h^T$ 。每个训练 token 都会以不同强度更新大量输出 rows。
- 对主干： $\partial L / \partial h = W_U^T (p - y_{\text{onehot}})$ 。输出空间把 V 维监督压回 d 维 residual state。

这解释了为什么 LM Head 不只是预测器：它还是监督信号的“投影器”。2026 年的最新工作进一步从优化角度研究这个投影，并提出 LM Head 可能形成 gradient bottleneck；该结论仍属新兴研究，需要更多独立复现。[R17]

4.1 稀有 token 的梯度并不等于“只在出现时更新”

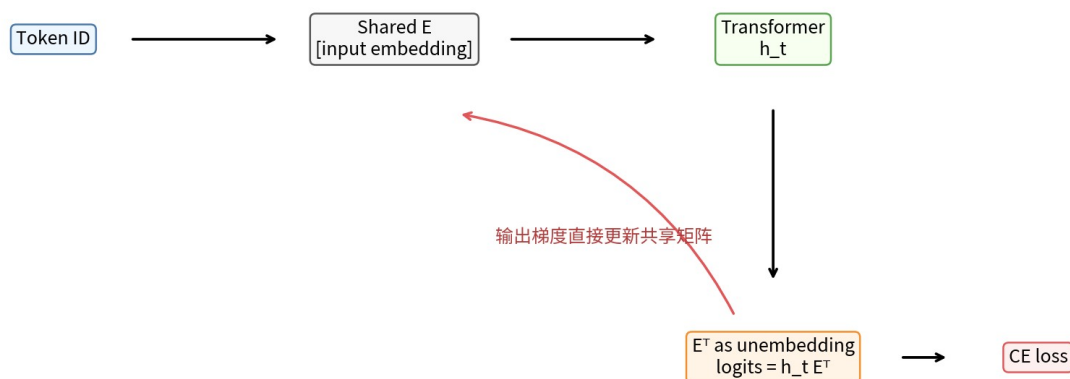
对于完整 softmax，即使某个 token 不是 target，它仍可能通过 p_i 获得梯度。因此 output embedding rows 与 input embedding rows 的训练统计非常不同：输入 row 通常只在 token 被 lookup 时收到直接梯度，而输出 row 理论上在每个位置都参与竞争。这个差异是理解 weight tying 的关键。

5. Weight Tying : 经典动机、收益与代价

经典 weight tying 设置:

$$W_U = E \quad \text{或} \quad \text{logits} = h E^T$$

Press & Wolf 以及 Inan 等工作在 2016-2017 年系统讨论了共享 input/output embeddings。Press & Wolf 观察到 output embedding 本身是有意义的词表示, 并推荐 tying, 以减参且在多种语言模型上改善 perplexity。[R1][R2]



Weight tying 减少 $V \times d$ 参数, 但把“读 token”和“预测 token”两个目标强耦合。

图 3 Weight tying 后, 同一矩阵同时承担 token 读取与 token 预测

维度	Tied	Untied
参数	节约 $V \times d$	额外付出 $V \times d$
输入/输出几何	强制共享	可分别适配
梯度	lookup 梯度 + output dense 梯度共同更新	两套梯度目标分离
小模型/大词表	减参收益通常更明显	参数成本更高
灵活性	较低	更高: 可不同维度/初始化/正则

5.1 为什么 classic tying 可能“偏向输出空间”

2026 年一项机制研究报告, tied embeddings 更接近 untied 模型的 output matrix, 而非 input embedding; 作者将其与训练早期 output gradients 占优联系起来, 并通过缩放输入梯度进行因果验证。[R15] 这是对经典 tying 直觉的重要补充, 但仍应视为最新研究, 而不是所有架构都已验证的定律。

5.2 Tying 绝对不是 transpose 就万事大吉

- Tokenizer 扩词表后, embedding 与 LM Head 必须同步 resize; 如果只改一侧, checkpoint 或 logits mapping 会失配。
- 若模型配置标记 tie_word_embeddings, 但保存/加载代码实际复制了权重而非共享 storage, 训练中可能悄悄解耦。
- 量化/分片时, shared parameter 的 alias 关系可能被破坏。

- 输入 embedding 常有 scale (如 $\sqrt{d}$) 或其它处理, 不代表输出端也应对 logits 做同样 scale。

5.3 新兴替代：更柔性的耦合

2026 年出现了 Output Embedding Centering、Pseudo-Inverse Tying 等新方法, 试图稳定输出几何或让 input/output interface 更一致。[R14][R16] 它们很有研究价值, 但尚不应视为通用生产默认。

6. Softmax Bottleneck 与表示能力

标准 LM Head 的 logits 是 $H W_U^T$ 。若收集大量 context hidden states 形成矩阵 H ，则 logits matrix 的 rank 至多受 d 限制。Yang 等把语言建模写成矩阵分解问题，并指出单一 softmax 参数化可能存在 “softmax bottleneck”。[R4]

$$\text{rank}(H W_U^T) \leq d$$

在需要表示高度多模态、上下文依赖强的 next-token distributions 时，目标 log-probability matrix 可能要求更高 rank。Mixture of Softmaxes 等方法通过多个 softmax components 增强表达能力。

重要边界： Softmax bottleneck 是数学表达能力限制，不等于 “任何现代大模型都因为它而明显性能受限”。后续研究指出某些理论不可达现象在实践中可能较少出现；2026 年也有工作从 “top tokens 概率是否真正受限” 重新审视该问题。

6.1 Bottleneck 与小模型

当 d 很小、 V 很大时，output matrix 的低秩约束相对更强。2024 年小模型研究重新讨论了模型饱和与 softmax bottleneck 的关系；2026 年又出现 “LM Head gradient bottleneck” 的优化视角。[R17] 因此 LM Head 可能同时存在 expressivity 与 optimization 两层问题，但二者不能混为一谈。

7. 大表：Vocabulary Size × Hidden Size 的系统权衡

增大 vocabulary 往往减少同一文本的 token 数，但 LM Head / embedding 成本线性增长：

$$\text{Parameters} \propto V d, \quad \text{projection FLOPs} \propto \text{tokens} \times V d$$

因此“更大的 vocabulary 更高效”不是无条件命题。Tokenizer 压缩率提升减少 L；LM Head 则随 V 增长。真实总成本取决于：

- 平均序列长度缩短多少；
- 模型 hidden size 与层数；
- 是否 weight tying；
- 训练时是否完整 vocab CE；
- 推理中 prefill/decode 比例；
- Tensor Parallel、memory bandwidth 与 kernel shape；
- 多语言/代码字符覆盖带来的质量收益。

2024 年关于 vocabulary scaling 的研究提示，大词表在某些 LLM 设置中可以提升性能，但最优点必须与模型大小、数据和计算预算联合搜索。[R18]

7.1 一个简单的 break-even 思维实验

假设 vocabulary 从 32K 增到 128K，使平均 token 数降低 25%。Transformer 主干的大部分层 FLOPs 随 token count 降低，但 output projection 的 V 变为 4 倍。最终是否更快取决于主干计算占比与 LM Head 计算占比，不能只比较“每字符 tokens”。

8. Adaptive / Hierarchical / Sampled Softmax

大词表问题在 LLM 之前就存在。历史上出现了多种减少 full softmax 线性 V 成本的方法：

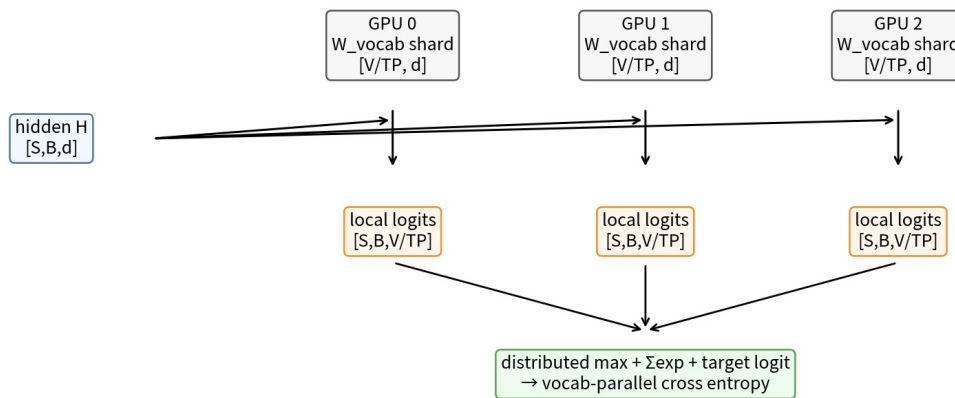
方法	核心理念	优点	主要代价 / 现代 LLM 位置
Hierarchical Softmax	把词表组织成树，预测路径	复杂度可近似 $O(\log V)$	树结构影响质量；GPU dense GEMM 利用率较差
Importance / Sampled Softmax	训练只对 target + 采样 negatives 计算	降低训练输出成本	概率校正、采样偏差；现代 subword V 通常没大到必须使用
Adaptive Softmax	频繁词大容量、稀有词分 cluster/低维	利用 Zipf 分布降低期望计算	架构复杂；更适合超大 word vocab
NCE / Self-normalization	把归一化问题转成判别或鼓励 $Z \approx \text{常数}$	可避免完整归一化	概率校准/训练目标差异
Full Softmax + TP	保留精确目标，按 vocab 分片	现代 GPU/分布式实现简单且精确	仍需 $V \times d$ 参数与跨 rank reduction

Adaptive Softmax 利用词频极不均衡，把常见词放在 head，稀有词放入 clusters，目标是最小化期望计算时间。[R3] Jean 等则通过 importance sampling 处理 very large target vocabulary。[R7]

Know-how: 现代 subword LLM 里，优先把 full softmax + fused / vocab-parallel CE 当作 baseline。只有当 V 、硬件或任务使 output layer 成为实测瓶颈时，再考虑近似方案；不要因为“理论上 $O(V)$ ”就自动引入复杂近似。

9. 分布式：Vocabulary Parallel LM Head

当 W_U 太大时，可沿 vocabulary 维切分：每个 Tensor Parallel rank 只保存 V/TP 行。



关键：无需 all-gather 完整 $[S, B, V]$ logits 才能算精确 cross-entropy。

图 4 Vocabulary-parallel LM Head 与分布式 cross-entropy

关键是：精确 cross-entropy 并不要求 all-gather 完整 logits。Megatron Core 的 VocabParallelCrossEntropy 直接在分片 logits 上计算全局 max、目标 logit、sum-exp 与梯度。[R9][R12]

9.1 分布式 logsumexp 的三步

1. 各 rank 计算 local max，做 all-reduce MAX 得到 global max。
2. 本地计算 $\exp(\text{logits} - \text{global_max})$ 的和，all-reduce SUM 得到 global denominator。
3. 只有拥有 target vocab id 的 rank 读取 target logit，再 reduce 得到全局 target logit。

这样通信量与 $[B, L]$ 标量统计相关，而不是把 $[B, L, V]$ 全部聚合。

9.2 高频 Gotchas

- V 不能整除 TP 时，需要 padding vocab；padding rows 必须在 loss/采样中正确屏蔽。
- weight tying 时 input embedding 和 output head 应使用兼容的 vocab sharding，否则共享关系与 checkpoint conversion 复杂。
- label smoothing 在 vocab-parallel CE 中必须基于全局 V 定义，不是 local V 。Megatron API 显式支持该参数。[R12]
- logits max 与 sum-exp reductions 必须保持足够精度；半精度直接累加大 V 可能放大误差。

10. 推理 : logits 、 Temperature 、 Top-k/Top-p 与实际成本

自回归 decode 每一步只需要最后位置 h_t ，但仍要把 h_t 投影到 vocabulary logits：

$$[B,d] \times [d,V] \rightarrow [B,V]$$

相比 prefill 的 $[B,L,V]$ ，decode 的输出 projection 张量更小；然而 V 很大时，每个 token 都要读 W_U ，因此可能表现出 memory-bandwidth 特征。

10.1 Softmax 并不一定需要对全部 logits materialize 概率

若只做 greedy argmax，严格来说不需要 softmax，因为 $\text{argmax}(z) = \text{argmax}(\text{softmax}(z))$ 。Top-k 也可以先在 logits 上找候选，再对候选做归一化；Top-p 则通常需要足够完整的排序/概率质量信息。不同 serving kernel 会采用不同 fusion。

10.2 Logit processors 的顺序非常重要

- repetition penalty / bad-word mask / min-length EOS mask 通常作用在 logits。
- temperature 应在 sampling distribution 构造前应用。
- top-k/top-p 的先后顺序会改变最终候选集合。
- 若使用 logit soft-capping，必须确认训练与推理是否一致。

Gotcha: 不要对已经 softmax 的概率再次按 “logit processor” 公式处理；许多 decoding 算法假设输入是未归一化 logits。

11. 数值稳定：Logit Drift、Soft-capping、z-loss、OEC

标准 stable softmax 可以避免直接 exp overflow，但不代表训练过程中 logits 可以无限增长而没有副作用。极端 logit scale 会带来：低精度分辨率问题、梯度极端集中、calibration 变化以及后续 sampling/蒸馏敏感性。

11.1 Gemma 2 的 final logit soft-capping

Gemma 2 在 final layer 使用：

$$z \leftarrow \text{cap} \times \tanh(z / \text{cap})$$

技术报告给出的 final logits soft_cap 为 30.0，并把该机制作为训练稳定化设计之一。[R11]

11.2 z-loss 的思想

LLM 工程里常见的 z-loss 与 2016 年 “Z-loss classification loss” 不是完全同一个概念。现代训练 recipe 中常用的 logit z-regularization 通常惩罚 log-partition：

$$L_z \propto (\log \sum_j \exp(z_j))^2$$

其目标是抑制 log-normalizer / logits 绝对尺度漂移。ST-MoE 把类似思想用于 router logits，并指出它源自 final softmax logits 的稳定化实践。[R13]

11.3 2026 Output Embedding Centering

OEC 工作把 output logit divergence 归因于 output embedding geometry 的 anisotropy，并提出 μ -centering / μ -loss。论文报告它们在其实验中比 z-loss 更稳定、与 logit soft-capping 相当。[R14] 这是值得跟踪的新方向，但还不是跨架构验证充分的默认 recipe。

12. Mixed Precision / FP8 / Quantization

LM Head 是大 GEMM，非常适合低精度加速，但 logits 与 cross-entropy 又对极值和归一化敏感。实务上常见策略是“低精度 GEMM + 更高精度 accumulation/reduction”。

位置	风险	Know-how
W_U GEMM	FP8/BF16 量化误差影响整个词表排序	监控 top-1/top-k parity、logit RMS/amax，而不只看平均 MSE
max reduction	错误 max 会影响所有 exp	global max 用稳定 dtype / 正确 TP reduction
sum exp	大 V 累加误差	FP32 accumulation 或经过验证的 fused kernel
target logit	边界 vocab shard / padding id 错位	做 reference CE parity test
量化 tied weight	输入 lookup 与输出 GEMM 的最佳量化方案可能不同	共享权重不等于共享量化 scale 必然最优

12.1 Weight tying 与量化的冲突

输入 embedding 是 row lookup，输出 unembedding 是大矩阵 GEMM；两者对量化误差与内核布局的敏感性不同。若一套共享权重必须服务两种 kernel，就需要确认 per-row / per-channel scale、layout transform 与 dequant 路径不会破坏共享语义或复制出隐形的第二份权重。

13. Special Tokens 、Tokenizer 与 LM Head 联合 Gotchas

LM Head 的输出 index 与 tokenizer vocabulary 是严格一一对应的 ABI。只要 token-id mapping 改变，输出语义就改变。

场景	典型错误	后果
新增 special token	只 resize embedding，不 resize LM head	目标 id 超界或新 token 永远无法预测
重排 vocabulary	checkpoint 权重行未同步重排	模型“说错字典”：logit index 映射到错误 token
PAD token	参与训练 target 或 sampling	模型浪费概率质量预测 padding
EOS/BOS	chat template 与训练 target 边界不一致	结束行为异常、重复 EOS/BOS
unused reserved tokens	未屏蔽却大量存在	可能形成 glitch / undertrained rows
多语言扩词表	新 rows 初始化太大/太小	初期新 token 概率异常、旧 token 分布被扰动

13.1 为什么“扩词表初始化为均值”有理论直觉

若新 output rows 随机初始化后与某些 hidden states 产生异常大 dot product，新 token 即使未训练也可能抢占概率质量。把新 rows 初始化到旧 embedding 分布中心或由旧 subwords 组合，通常更保守。但最优初始化依赖 tying、normalization 与 continued-pretraining 数据；应通过短程训练验证，而不是只看初始化时 loss。

14. 50 个 Gotchas 清单

01. 把 LM Head 当成“无成本 Linear”，忽略 $V \times d$ 参数与训练 FLOPs。
02. 显式 softmax 后再 log 计算 CE，增加数值风险和内存。
03. 忘记 logits 的 shift invariance，但又误以为绝对 logit scale 永远无关紧要。
04. 用 local vocab size 做 label smoothing。
05. vocab parallel 时先 all-gather 全 logits，失去分片 CE 的主要收益。
06. V 不整除 TP 时 padding rows 没有正确 mask。
07. checkpoint 转换时 vocab shard 顺序错位。
08. tokenizer token-id 重排但 LM Head rows 未重排。
09. 扩词表只 resize input embedding，未处理 output head。
10. 配置声称 tied，但 runtime storage 实际不共享。
11. 量化转换后 tied parameter 被复制成两份且逐渐失配。
12. 误把 tied E 与 E^T 当作严格可逆编码/解码。
13. 忽略 input lookup 梯度与 output dense gradient 的统计差异。
14. 默认 tying 在任何模型规模都更好。
15. untie 后忘记重新评估初始化与学习率。
16. 用旧模型的 LM Head 初始化新 tokenizer，但 token 语义不对应。
17. 随机初始化新 vocab rows 产生异常大 logits。
18. special token 的 output row 没有得到足够训练。
19. PAD token 进入 CE target。
20. 训练时 ignore_index 与 tokenizer pad_id 不一致。
21. EOS 在 packing boundary 被错误屏蔽或重复计算。
22. loss shift 写错：input/target 没有错一位。
23. 把 perplexity 按 padding token 一起平均。
24. sequence packing 后跨文档 boundary 的 target 未按 recipe 处理。
25. CrossEntropy reduction 的 denominator 使用 batch size 而不是有效 token 数。
26. 梯度累积时 loss normalization 不一致。
27. BF16/FP16 sum-exp reduction 精度不足。
28. logit amax 持续增长却只监控 total loss。
29. 为了稳定直接硬 clip logits，改变排序/梯度却未做 ablation。
30. 训练使用 soft-capping，推理忘记应用。
31. 推理额外重复 soft-capping。
32. temperature=0 直接做除法而不是走 greedy 分支。
33. top-p 前把 logits 当概率。
34. 先 top-p 再应用 banned-token mask，导致质量不守恒。

- 35. greedy decoding 仍 materialize 完整 softmax probability, 浪费带宽。
- 36. 大词表 decode 中只优化 attention kernel, 忽略 output GEMM。
- 37. 误认为 adaptive softmax 在现代 GPU 下一定比 full softmax 快。
- 38. sampled softmax 没有正确 importance correction。
- 39. 用 approximate output layer 训练, 评测却假设精确归一化概率。
- 40. softmax bottleneck 理论结论被夸大成所有失败的原因。
- 41. 把 expressivity bottleneck 与 gradient bottleneck 混为一谈。
- 42. weight tying ablation 参数量不匹配, 导致比较不公平。
- 43. 不同 vocab size ablation 没有控制训练字符/字节预算。
- 44. 只控制 token 数比较 tokenizer, 导致原始文本量不同。
- 45. 大词表模型的 embedding/head 参数未计入 “non-embedding model size” 比较。
- 46. 分布式 CE 的 global max reduction 使用错误 process group。
- 47. vocab-parallel target ownership mask off-by-one。
- 48. checkpoint vocab padding size 与 serving engine 不一致。
- 49. 输出 bias 的有无在 checkpoint 转换时被忽略。
- 50. 用 logit-lens 解释时忽略 final norm / tying / unembedding geometry 的差异。

15. Know-how : 从零实现与生产检查清单

15.1 From-scratch 最小正确实现顺序

4. 先实现 FP32 reference: `logits = F.linear(h, W, bias)`; `loss = cross_entropy(logits.view(-1,V), target.view(-1), ignore_index=pad)`。
5. 做 shift test: 模型输入 `x[:, :-1]` 对应 target `x[:, 1:]`, 确认第一/最后 token 边界。
6. 对随机小张量, 用手写 `logsumexp` CE 与框架 CE 做 forward/backward parity。
7. 决定 tied vs untied, 并用 object identity / `data_ptr` 验证真共享, 而不是数值相等。
8. 接入 mixed precision 前记录 FP32 baseline 的 loss、grad norm、logit RMS/max、top-k。
9. 再做 fused CE / vocab-parallel CE; 每一步都与 FP32/full-vocab reference 对齐。
10. 最后才引入 soft-capping、z-loss、quantization 等稳定化/效率技巧。

15.2 训练前 Checklist

- vocab_size 与 tokenizer length 完全一致
- special token IDs 与 config 一致
- weight tying 符合设计且真实共享
- embedding/head init scale 合理
- ignore_index 与 padding 策略一致
- loss denominator 是有效 target token 数
- TP vocab padding 与 target mapping 正确
- FP32 reference parity 通过
- checkpoint save/load 后 tying 与 vocab mapping 不变

15.3 Serving 前 Checklist

- serving tokenizer 与 checkpoint vocab hash/版本一致
- logit processor 顺序固定并测试
- 训练使用的 final logit soft-capping 在 serving 一致
- greedy 路径避免不必要 softmax
- TP shard 与 checkpoint conversion 正确
- quantized LM Head top-k parity 可接受
- EOS/PAD/BOS mask 与 chat template 一致
- speculative decoding 的 draft/target vocab mapping 兼容

16. Know-why : 20 核心因果原理

问题	Know-why
为什么 W_U 是 $V \times d$?	每个 token 需要一个从 d 维 hidden state 到标量 logit 的线性分类方向。
为什么 full softmax 是 $O(V)$?	归一化需要所有类别的 exp 总和; full projection 本身也产生 V 个 logits。
为什么 stable softmax 减 max?	softmax 对共同平移不变, 减最大值把最大指数变成 $\exp(0)=1$, 避免溢出。
为什么 CE 梯度是 $p-y$?	softmax Jacobian 与 $-\log p_y$ 链式求导后恰好化简。
为什么 output rows 都会被更新?	非 target row 仍有 p_i 项; 完整 softmax 是全类别竞争。
为什么 tying 特别适合小模型?	$V \times d$ 在总参数中占比更高, 省掉一份 head 的相对收益更大。
为什么 tying 可能偏向输出任务?	输出端每个 token 都参与 dense competition, 而输入端是稀疏 lookup; 梯度统计不对称。
为什么 softmax bottleneck 与 d 有关?	logits matrix $= H W^T$, 其 rank 受中间维度 d 上界约束。
为什么大词表可能减少主干 FLOPs?	同一文本被编码成更少 tokens, 所有按 L 计算的层都会受益。
为什么大词表又增加 head 成本?	W_U 行数就是 V , 参数和每位置 projection 都线性随 V 增长。
为什么 vocab parallel 沿 V 切最自然?	每个 token 的分类 rows 可独立分片; 只需全局 max/sum/target 统计即可精确 CE。
为什么 greedy 不需要 softmax?	softmax 是严格单调变换, 保持 argmax。
为什么 top-k 可以先在 logits 上做?	排序同样被 softmax 保持。
为什么 top-p 更依赖归一化?	阈值定义在累积概率质量上, 需要概率或等价归一化信息。
为什么 logit soft-capping 用 tanh?	平滑地限制绝对值, 同时保持中心区域近似线性, 避免硬 clip 的不可导拐点。
为什么 z-loss 能约束 scale?	logsumexp 是 softmax 的 log-partition, 惩罚它可抑制共同 logit 漂移。
为什么量化 LM Head 要看 top-k parity?	最终任务依赖 token 排序与相对 gap, 平均权重误差直接等价于解码质量。
为什么新 token 初始化会影响旧 token?	softmax 分母包含所有 rows, 新 row 即使不是 target 也会竞争概率质量。
为什么 tokenizer 与 LM Head 是 ABI?	token id 是 W_U row 的语义地址; 任何映射变化都会改变输出含义。
为什么 LM Head 值得独立监控?	主干 loss 看似正常时, logit scale、row norm、rare-token gradients 仍可能已出现异常。

17. Monitoring 与 Debug Playbook

17.1 建议监控指标

指标	意义	异常信号
logit mean / RMS / max	输出尺度	持续单调增长、突变
logsumexp / partition	softmax normalization scale	大幅漂移、z-loss 同步上升
target logit vs max logit gap	正确 token 排序余量	长期为负或方差异常
entropy	预测尖锐度	过早塌缩或异常高熵
W_U row norm 分布	token classifier geometry	少数 rows 爆炸、频率相关极端偏差
grad norm: W_U / embedding	输出与输入梯度平衡	tied 模型 output dominance
rare-token NLL	长尾学习	整体 loss 降但长尾恶化
top-k token churn	数值/量化稳定	小数值变化造成候选剧烈跳变
TP CE parity	分布式正确性	单卡和多卡 loss/grad 不一致

17.2 NaN / Loss Spike Debug 顺序

- 11. 检查输入 targets 是否越界、是否存在错误 special IDs。
- 12. 记录 spike 前后 logit max、logsumexp、W_U norm、hidden RMS。
- 13. 切到 FP32 CE / 关闭 fused kernel，判断数值还是模型动态。
- 14. 若 TP：对比 full-vocab 单卡 reference，检查 global max/sum 与 target ownership。
- 15. 若 tied：分别记录 input-side 与 output-side 对共享矩阵的梯度贡献。
- 16. 关闭 soft-capping/z-loss/量化等附加机制做最小化定位。
- 17. 检查优化器 state / LR spike / checkpoint resume 是否改变共享参数关系。

18. Ablation / Benchmark Matrix

Ablation	必须控制	观察指标	为什么
Tied vs Untied	总参数或主干宽度匹配、初始化、token budget	PPL、长尾 NLL、embedding geometry、训练吞吐	区分参数量收益与耦合本身
Vocab 32K→128K	原始字符/字节数据量、训练 FLOPs	tokens/byte、PPL、LM-head FLOPs、端到端吞吐	防止 token budget 比较作弊
Full vs Adaptive Softmax	相同训练数据/主干/目标	wall-clock、概率校准、rare-token 质量	评估真实速度-质量前沿
Soft-cap on/off	LR/seed/precision	logit amax、loss spikes、final score	确认稳定收益是否有质量代价
FP8 head vs BF16	同一 hidden state 输入	top-k parity、CE 差异、吞吐	定位输出层量化误差
Vocab TP size	global batch/parallel 总规模	CE parity、通信量、吞吐	验证 shard / reduction 开销
新 token init	相同 continued-pretrain budget	初始 loss、短程 loss、旧域回归、新 token NLL	避免只看 step-0 loss

19. 2024-2026 新研究与开放问题

以下内容是值得跟踪的前沿方向。报告将它们与成熟工程实践分开，避免把单篇近期预印本直接升级为默认 recipe。

方向	2026 观察	成熟度判断
Output Embedding Centering	把 logit divergence 与 output embedding anisotropy 联系，提出 μ -centering / μ -loss。[R14]	新兴；需要更多规模/架构复现
Weight Tying Bias	报告 tied embedding 被 output gradients 主导，更接近 output space。[R15]	新兴机制解释；对 tying 选型有启发
Pseudo-Inverse Tying	显式构造 input/output 的稳定耦合接口。[R16]	探索性结构；尚非主流
LM Head Gradient Bottleneck	把 softmax bottleneck 扩展为监督反传的信息压缩问题。[R17]	非常新；结论强，需要独立验证
Large Vocabulary Scaling	研究 V 与模型性能/效率共同缩放。[R18]	方向较稳，但最优 V 强依赖数据与硬件

19.1 仍未解决的核心问题

- 是否存在随模型规模与 token budget 可预测的最优 vocabulary/head 参数比例？
- weight tying 的收益何时从“正则 + 减参”转为“几何冲突 + 梯度不平衡”？
- 能否设计兼容大词表、精确概率、GPU dense kernel 的更强 output parameterization？
- softmax bottleneck 在超大 Transformer 的真实瓶颈程度究竟有多大？
- LM Head 的低精度、稀疏化与 candidate retrieval 能否真正降低 decode latency，而不破坏概率质量？

20. PyTorch Reference Implementation

下面代码强调“最小可验证”，不是追求最高性能。生产环境应使用框架 fused CE / Tensor Parallel 实现。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TinyLMHead(nn.Module):
    def __init__(self, vocab_size, d_model, embedding=None, tie=True):
        super().__init__()
        if tie:
            assert embedding is not None
            self.weight = embedding.weight # true shared Parameter
        else:
            self.weight = nn.Parameter(torch.empty(vocab_size, d_model))
            nn.init.normal_(self.weight, mean=0.0, std=d_model ** -0.5)

    def forward(self, h):
        # h: [B, L, d]
        return F.linear(h, self.weight) # [B, L, V]

def causal_lm_loss(logits, input_ids, ignore_index=-100):
    # predict x[t+1] from hidden state at t
    pred = logits[:, :-1, :].contiguous()
    tgt = input_ids[:, 1:].contiguous()
    return F.cross_entropy(
        pred.view(-1, pred.size(-1)),
        tgt.view(-1),
        ignore_index=ignore_index,
    )
```

20.1 手写 Stable CE Parity Test

```
def reference_ce(logits, target):
    # logits: [N,V], target: [N]
    m = logits.max(dim=-1, keepdim=True).values
    lse = m.squeeze(-1) + torch.log(torch.exp(logits-m).sum(dim=-1))
    zy = logits.gather(1, target[:,None]).squeeze(1)
    return (lse - zy).mean()

torch.manual_seed(0)
z = torch.randn(8, 17, dtype=torch.float64, requires_grad=True)
y = torch.randint(0, 17, (8,))
a = reference_ce(z, y)
b = F.cross_entropy(z, y)
```

```
print((a-b).abs().item())
```

20.2 Tying 真共享验证

```
assert model.lm_head.weight is model.embed_tokens.weight  
assert model.lm_head.weight.data_ptr() == model.embed_tokens.weight.data_ptr()
```

21. 25 道深度面 / 推导题

1. 推导 cross-entropy 对 logits 的梯度为什么等于 $p-y$ 。
2. 推导 $\partial L / \partial h$ ，并解释 LM Head geometry 如何影响主干监督方向。
3. 为什么 softmax 可以减 max 而概率不变？
4. 为什么 greedy decoding 不需要 softmax？
5. Top-k 为什么可在 logits 上做，Top-p 为什么更麻烦？
6. 一个 $V=256K$ 、 $d=4096$ 的 untied LM Head 有多少参数？BF16 权重约多少显存？
7. 为什么 weight tying 对小模型通常更有参数价值？
8. 输入 embedding 与 output embedding 的梯度统计为什么不同？
9. 解释 softmax bottleneck 的 rank 上界。
10. Mixture of Softmaxes 如何突破单一 softmax rank 限制？
11. 为什么大 vocabulary 既可能更快又可能更慢？
12. 设计一个公平的 32K vs 128K tokenizer+LM Head ablation。
13. Adaptive Softmax 为什么利用 Zipf 分布？
14. 为什么现代 LLM 更常见 full softmax + vocab parallel，而不是 hierarchical softmax？
15. 推导 vocab-parallel cross entropy 的 global max / sum-exp 流程。
16. V 不能整除 TP 时如何安全 padding？
17. 训练 loss 突然 spike，但 hidden RMS 正常、logit max 暴涨，你先查什么？
18. logit soft-capping 为什么用 tanh 而不是 hard clamp？
19. 解释现代 z-loss 与 2016 Z-loss 论文命名容易混淆的地方。
20. tied model 转 untied checkpoint 时你会如何初始化 output head？
21. 扩 10K 新 tokens 时，为什么随机大方差初始化会扰动旧 token 概率？
22. 量化 LM Head 应该用哪些 task-aware parity 指标？
23. 为什么 logit-lens 分析依赖 unembedding geometry？
24. 如何测试 serving tokenizer 与 LM Head row mapping 完全一致？
25. 如果未来要把 LM Head 替换成 approximate candidate retrieval，你认为最大风险是什么？

22. 参考文献（优先原始论文 / 官方文档）

- [R1] Press, O. & Wolf, L. Using the Output Embedding to Improve Language Models. arXiv:1608.05859 (2016/2017).
<https://arxiv.org/abs/1608.05859>
- [R2] Inan, H., Khosravi, K., Socher, R. Tying Word Vectors and Word Classifiers: A Loss Framework for Language Modeling. arXiv:1611.01462.
<https://arxiv.org/abs/1611.01462>
- [R3] Grave, E. et al. Efficient Softmax Approximation for GPUs (Adaptive Softmax). arXiv:1609.04309.
<https://arxiv.org/abs/1609.04309>
- [R4] Yang, Z. et al. Breaking the Softmax Bottleneck: A High-Rank RNN Language Model. arXiv:1711.03953.
<https://arxiv.org/abs/1711.03953>
- [R5] Baevski, A. & Auli, M. Adaptive Input Representations for Neural Language Modeling. arXiv:1809.10853.
<https://arxiv.org/abs/1809.10853>
- [R6] Chen, W., Grangier, D., Auli, M. Strategies for Training Large Vocabulary Neural Language Models. arXiv:1512.04906.
<https://arxiv.org/abs/1512.04906>
- [R7] Jean, S. et al. On Using Very Large Target Vocabulary for Neural Machine Translation. arXiv:1412.2007.
<https://arxiv.org/abs/1412.2007>
- [R8] Shueybi, M. et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053.
<https://arxiv.org/abs/1909.08053>
- [R9] NVIDIA Megatron Core: VocabParallelCrossEntropy official documentation (accessed 2026-08-14).
https://docs.nvidia.com/megatron-core/developer-guide/0.15.0/apidocs/core/core.tensor_parallel.cross_entropy.html
- [R10] PyTorch CrossEntropyLoss official documentation (accessed 2026-08-14).
<https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
- [R11] Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118.
<https://arxiv.org/abs/2408.00118>
- [R12] NVIDIA Megatron Core tensor-parallel package documentation.
https://docs.nvidia.com/megatron-core/developer-guide/0.15.0/api-guide/tensor_parallel.html
- [R13] Zoph, B. et al. ST-MoE: Designing Stable and Transferable Sparse Expert Models. arXiv:2202.08906.
<https://arxiv.org/abs/2202.08906>
- [R14] Stollenwerk, F. et al. Output Embedding Centering for Stable LLM Pretraining. arXiv:2601.02031 (2026).
<https://arxiv.org/abs/2601.02031>
- [R15] Lopardo, A. et al. Weight Tying Biases Token Embeddings Towards the Output Space. arXiv:2603.26663 (2026).
<https://arxiv.org/abs/2603.26663>
- [R16] Gu, J. et al. Rethinking Weight Tying: Pseudo-Inverse Tying for LM Stable Training and Updates. arXiv:2602.04556 (2026).
<https://arxiv.org/abs/2602.04556>
- [R17] Lost in Backpropagation: The LM Head is a Gradient Bottleneck. arXiv:2603.10145 (2026).
<https://arxiv.org/abs/2603.10145>

[R18] Large Vocabulary Size Improves Large Language Models. arXiv:2406.16508 (2024).

<https://arxiv.org/abs/2406.16508>

注：2026 年文献为近期研究，报告只将其作为“新兴证据/开放方向”，没有把单篇结果当作成熟工程共识。